Video : **https://mediaspace.msu.edu/media/group12-5classes-overview/1_bd0tf220**

Johnny:
Hello, I am Johnny and I implemented the RobinHoodMap. The purpose of this class is to make a better hashmap than the standard library unordered_map. The way we make this more efficient is through Robin Hood Hashing.

What is Robinhood Hashing:
Robinhood hashing is a variant of linear probing in hash tables. The reason why this is different though is it tracks the distance from the original hashing location. By doing this it can become more efficient by finding out the least amount of probing possible. It does this by checking each element it probes distance from its original hash, it is lower than the current element we are adding, then swap and move the one already there further, making sure elements never need to probe too far from its original spot.

Not knowing Robinhood hashing may be confusing and I don't want my whole video to be on it so here are a couple resources to help you out.
https://www.geeksforgeeks.org/dsa/robin-hood-hashing/
https://www.cs.cornell.edu/courses/JavaAndDS/files/hashing_RobinHood.pdf
https://programming.guide/robin-hood-hashing.html

Main Functionality:
I included most functions that you would use in a hashmap, mostly inserting, searching, and deleting all have their own functions. However I also overloaded operators such as the brackets, asterisk, ++, - -, etc. to make it act as similar to hashmaps in python and C++ as I could.

Storage:
For the storage I used a struct called Entry which tracked the key, the value, whether this entry is filled, and the distance from its original probe. Then I used a vector to store these entries. I created a resize function as well so that we can make sure we resize the vector when it fills up. Specifically I chose 50% capacity, the reason why I did this is

because when doing research I found that high capacity in RobinHoodHashing actually didn’t hurt performance as much as it did other hashmaps.

Iterators:
I included 2 types of iterators, a const iterator and a regular iterator class for going through the map. Now an important piece of information I would like to stress is I did not handle a way to delete through iterators. This is intentional as it is very risky deleting an iterator in a robinhood map since it may or may not shift everything. I also included begin(), end(), cbegin(), and cend() functions.

Other Details:
I included quite a few private functions when dealing with inserting and deletion or resizing, if you see a function that starts with an underline that means it's private. I added a static_assert in a couple areas, such as in the * operator to make sure they cannot dereference the end iterator or too high of an index. I check that the key value is hashable at the start.

If you have any questions for me please do ask in the public group 12 chat. Thank you!

Cindy:
Hello, my name is Cindy. I implemented the DataMap class, which is a flexible container that lets you store values of different types using a string as an identifier. Internally, it uses an unordered map. The keys are strings and the values are of type `std::any`, which means each entry is mapped by name and can hold any type of value. The original type is still preserved, though.

The user can call the Set function to store entries into the DataMap. It takes a string and a value of any type as the parameters.

The user can call the Get function to retrieve a value from the DataMap. It takes a string identifier and the expected type of the value being retrieved. The function call should look like `Get<T>(std::string key)`. If the key doesn't exist or the requested type does not match the actual type of the value, an assert is triggered. If everything is correct, the function returns the value using `std::any_cast`.

The key idea of this class is flexibility. Instead of creating a ton of separate member variables or rigid structures, we can dynamically store and retrieve values of various types while still remaining type-safe.

Introduction:

Hi my name is Landon Cosby, I am a member of Group 12 and I was in charge of implementing the TagManager class. TagManager works closely with AnnotationSet to track objects and their associated tags. TagManager is required to allow fast querying of objects based on tags, it accomplishes this by using 2 maps.

How my class works:

Storage:
tag_index maps a tag to all objectID's that have that tag, while object_tags maps an objectID with all its tags. This two way indexing allows for fast lookups when being queried, for example when querying by a single tag then using tag_index I can immediately return all the objectID's with that tag. object_tags is useful anytime we want to remove objects as we can find what tags are related to the object then remove them directly from the tag_index saving us from searching every tag_index entry to find where the objectID is listed.

Query's:
Single tag queries are simple as the information needed is stored in tag_index, querying with multiple tags that must be included and not included is not so simple. QueryMultiTags takes two arguments, first a list of all the tags that must be included in the final result, and a list of tags that must not be included, which defaults to empty if not provided. When running the QueryMultiTags it first takes a list of all the objectID's that have the first tag required, then calls the AND helper function, this function searches through the set of objectID's in each tag that needs to be included and removes any id that is in the first tag set but not the others. It then checks to see if there exist any tags that it must not include, if there are then it calls the NOT helper function which removes any id's that show up in the tag_index set of the tags not to be included.

RegisterObject:
RegisterObject is in charge of taking an objectID and an annotation set and then storing them in the two maps where all connections are stored. It first checks to make sure the id is not a duplicate then runs through a loop to store all information needed. The UnregisterObject finds all tags related to objectID, uses these tags to quickly find and remove id's from tag_index, then deletes entry from object_tags.

Add/Remove/ClearTags:
AddTag takes the id, AnnotationSet, and new tag, it then updates both the AnnotationSet and tag_index/object_tags. RemoveTag takes the same arguments, but instead of adding to the maps it removes one tag. ClearTags takes the id and AnnotationSet, then removes all tag entries.

How you may want to use the class:

It will be useful for storing information about agents, items, or maybe parts of the world. For example, agents may want to store "aggressive", “infected”, “clean”, etc. To use this class you will need to assign unique numbers to objects, create an AnnotationSet for each object, and a TagManager for each group of objects that you may have, like one for all the agents and one for the items.

AnnotationSet Transcript:

Okay. My class that I made is the annotation set. Basically, what this annotation set would be used for, in a system like the one we're working in would be a light weight dynamic labeling system for objects. So, basically, without it would allow objects if it's for agents, items, locations, etc. to carry a descriptive tag without modifying its core class definitions. A few features in the annotation set are add tags so that would allow you to add a string tag to the unordered set. So in the annotationset.hpp there is an unordered_set and you can add tags, various strings, to the unordered set. It has remove tags so it will remove the tag when called so if your object has a descriptive string and you want to remove it, you can remove that tag. If you are looking for tags, it will check if your object has tags. So if you're looking for object or agents with a specific tag you can search for those tags in the annotation set. It has "gettag" so it will find the tag and return a pointer to the tag. It has "gettags" which returns the entire unordered set of tags. It has size, so that will return the size of how many tags are in the annotation set. It has "empty" which checks if the annotation set is empty. It also has "clear", which will clear out all the tags in the annotation set. I will be working on some more features, but for the most part this is the basic annotation set and what it might be used for in a project like ours.

Lewi Anamo - ExpressionParser Script

Introduction & Purpose:

Hi everyone. I implemented the `ExpressionParser` class, a module designed to turn mathematical and logical strings into reusable function objects.

How it Works: The Two-Stage Engine:

The class works in two distinct stages.

1. **Stage One: Lexical & Syntax Analysis.** When you set an expression, the parser immediately tokenizes the string. It identifies literals, named variables, and indexed data points—like `{0}` or `{1}`. A stack-based validator then runs to ensure the 'grammar' is perfect, catching mismatched parentheses or illegal sequences before any math happens.
2. **Stage Two: The Shunting-Yard.** The Shunting-Yard algorithm acts as a traffic controller that uses a stack to reorder traditional "infix" expressions (where operators sit between numbers) into a format that prioritizes mathematical precedence. The resulting Reverse Polish Notation (RPN) places operators after their operands, creating a linear "instruction list" that a computer can evaluate efficiently in a single pass without needing to handle parentheses.

Public Interface & Usage

"Using the class is straightforward. You instantiate the parser with your expression, then call `parse()` to receive a `std::function` object. You can then call this function by passing two optional containers:

- A `std::map` for named variables like `'hp'` or `'level'`.
- A `std::vector` for indexed data.

Because of the default arguments, you can provide just a map, just a vector, or a hybrid of both for expressions like `hp + {0}`."

Error Handling & Safety

"The module utilizes a custom `ExpressionException` class for error handling.

- **Critical Errors**, like syntax mistakes, are caught during the initial parse.
- **Recoverable Errors**, such as a missing variable name in your map or an index out of bounds, are caught during execution using `.at()` lookups. This allows the calling module to catch the error, prompt the user for data, and resume without a crash."

Conclusion:

"Thanks for watching."